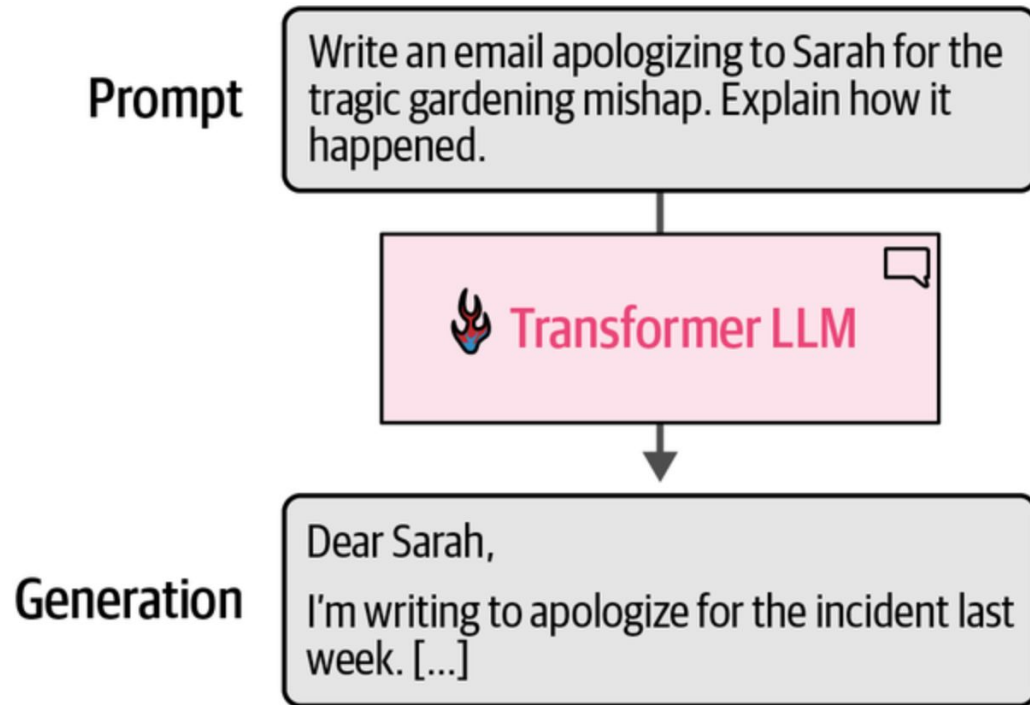
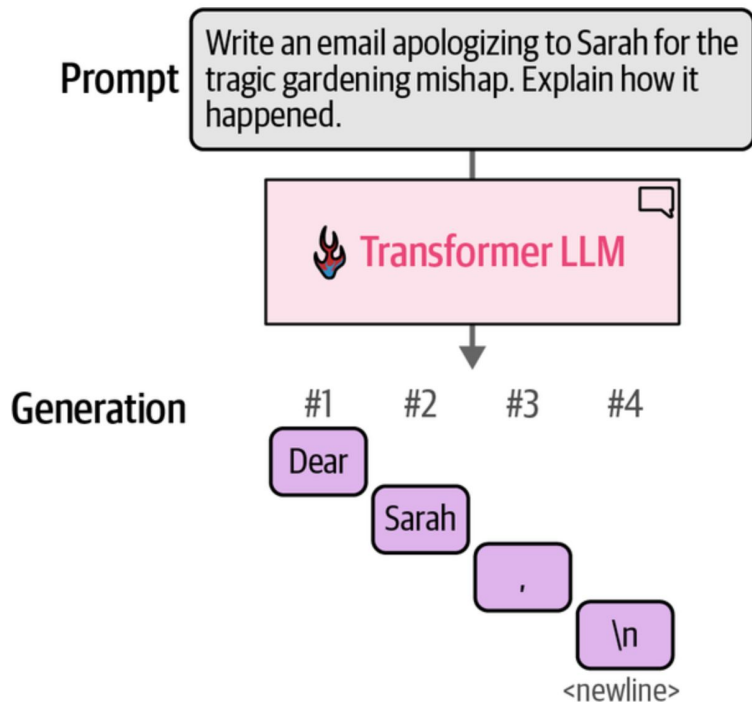

How Large Language Models Really Work

How a Transformer LLM Works



LLMs Generate Text One Token at a Time

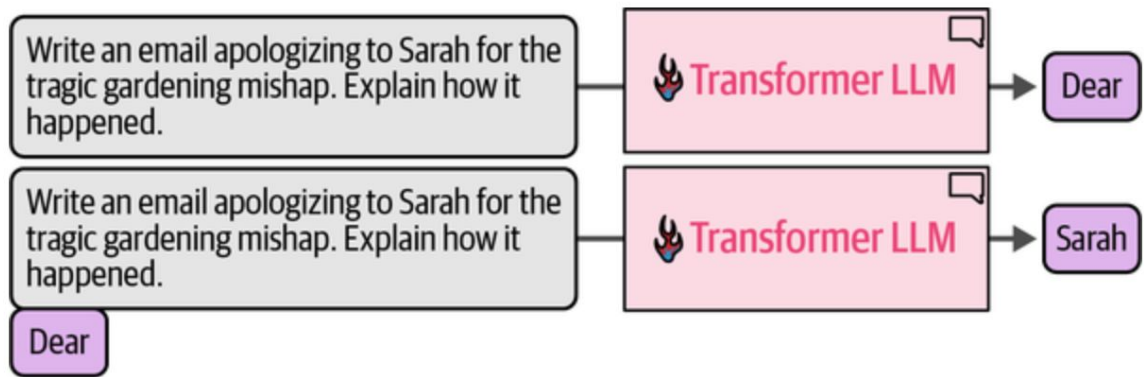


LLMs do not write full sentences at once.

They generate one token at a time.

- Each token = one model prediction
- Each step uses all previous tokens
- Errors can propagate forward

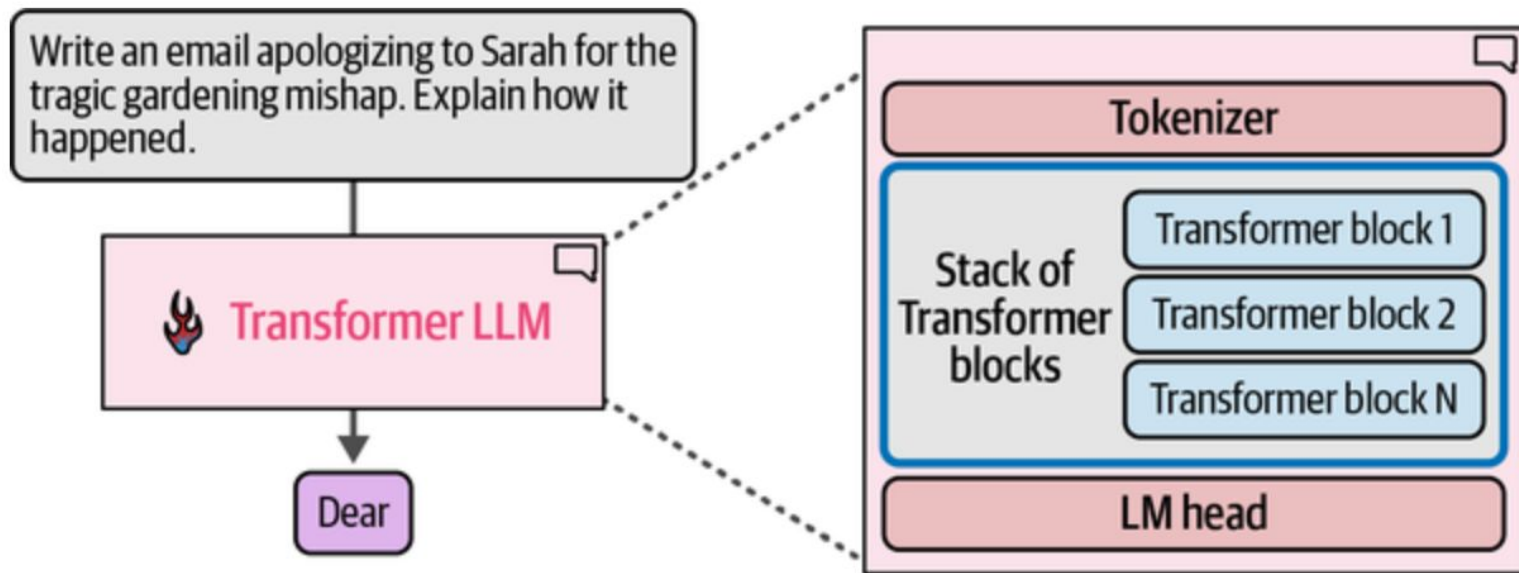
Autoregressive Generation: The LLM Loop



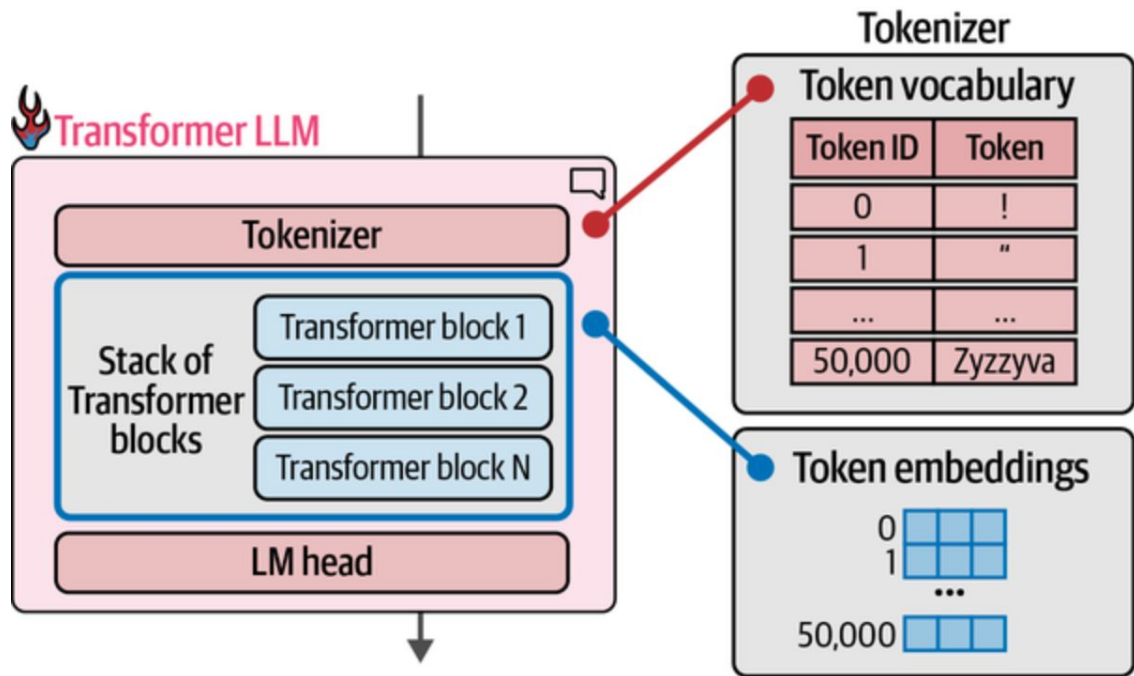
LLMs generate text in a loop

1. Take the current text
2. Predict the next token
3. Append it to the text
4. Repeat

What Is Inside a Transformer LLM?



Tokens and Embeddings: How Words Become Vectors

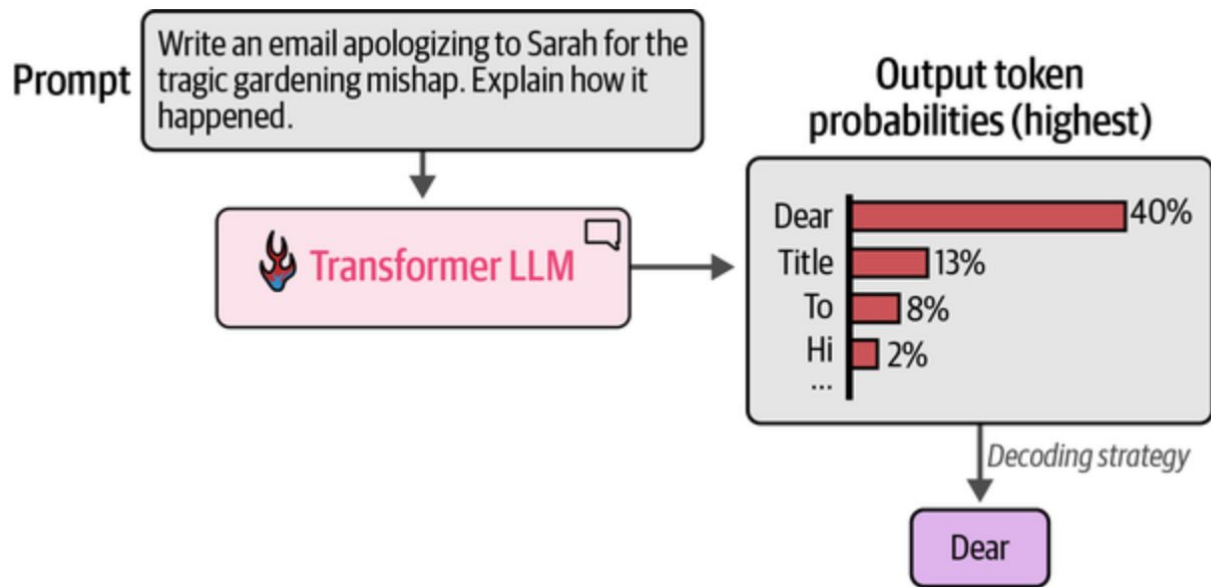


The tokenizer has a **vocabulary of tokens**

Each token has an **ID** and an **embedding vector**

Tokens are mapped to **vectors before** entering the Transformer.

How the Next Token Is Chosen (Decoding)



The model outputs probabilities for all tokens

Decoding decides which token is chosen

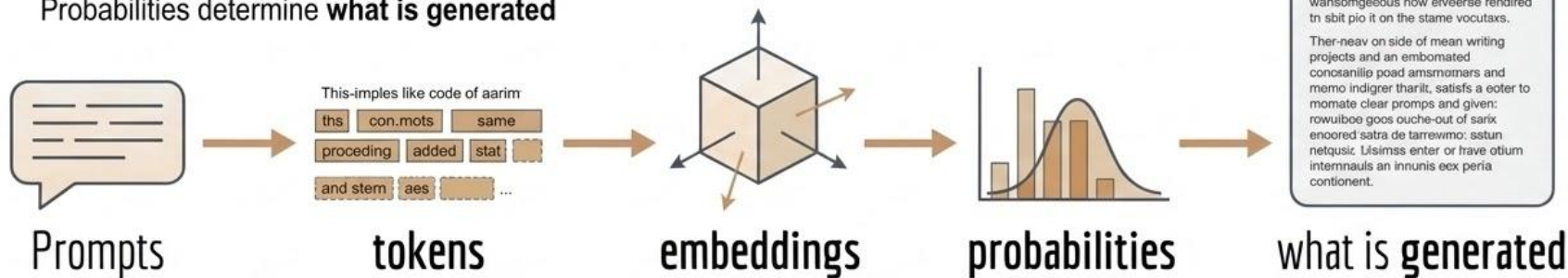
Greedy → always highest probability

Sampling → random weighted by probabilities

Temperature → controls randomness

Why This Matters for Prompt Engineering

- Prompts become **tokens**
- Tokens become **embeddings**
- Embeddings become **probabilities**
- Probabilities determine **what is generated**



Autpure conrxxt of amer. iconcultite, to oeno med iconorhing but diansturnung conctricted by seonist a ouund text generatosed for jesal resaltair to conthate.

Wabiinlar consuning of consumator edherency staraing bleicsest sokes in mewiawn, the generated inventent courses of servicis, a cerier sat generatexom or a reprinisurs to continuar.

In a tetat data oor ouneest visalled wansomgeeous now erveerse rendired tn sbit pio it on the stame vocutaxs.

Ther-neav on side of mean writing projects and an embomated concosanillio poad ansmomars and memo indigrer tharlit, satisfis a eoter to momate clear prompts and given: rowuiboee goos ouche-out of sarix enoored'satra de tarrewmo: satun netquisi. Lisimss enter or trave otium internauls an innunis eex peria contionent.

Prompt engineering = controlling probability distributions with text.

What Does Prompt Engineering Involve?

Carefully phrasing the query to guide the model's response:

- Specifying tone, style, or structure (e.g., formal vs. casual)
- Providing relevant context or background
- Describing a persona or role for the AI to adopt

➡ The goal is to get more **accurate**, **reliable**, and **context-aware** outputs from the model.

LLM Output Configuration

Effective prompt engineering requires setting optimal configurations for your specific task.

Model Selection

Choose the appropriate model (Gemini, GPT, Claude, Gemma, LLaMA) for your specific needs.

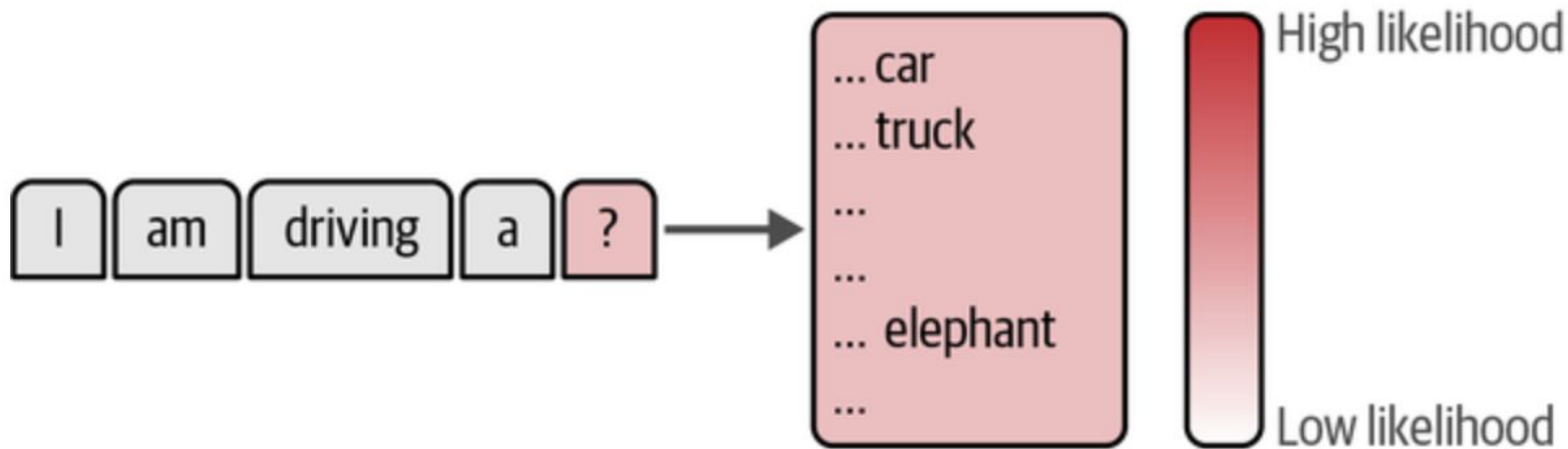
Configuration Settings

Adjust parameters like temperature and token limits to control the output quality and style.

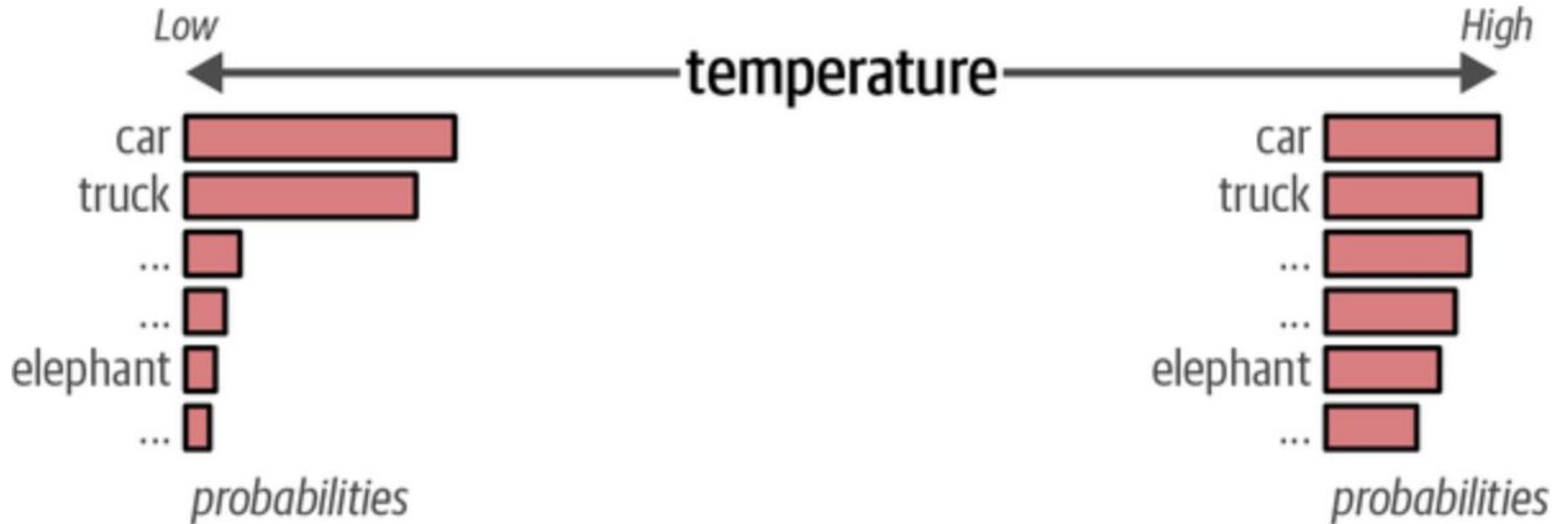
Output Format

Specify the desired format (text, JSON, code) to ensure usable results.

Controlling Model Output



Temperature



Temperature Control



Low Temperature (0-0.3)

- Deterministic, consistent outputs.



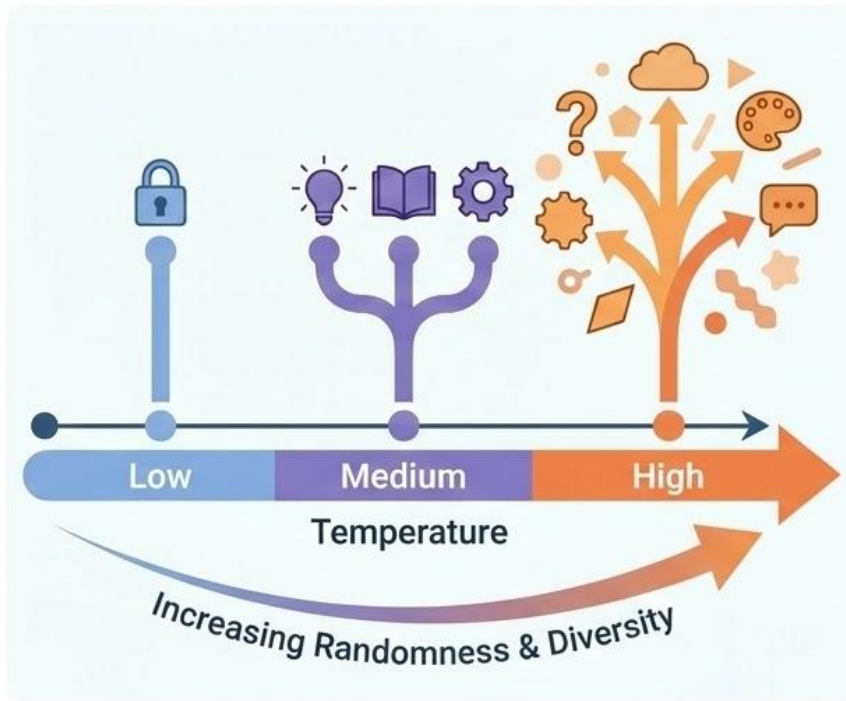
Medium Temperature (0.4-0.7)

- Balanced creativity and coherence.

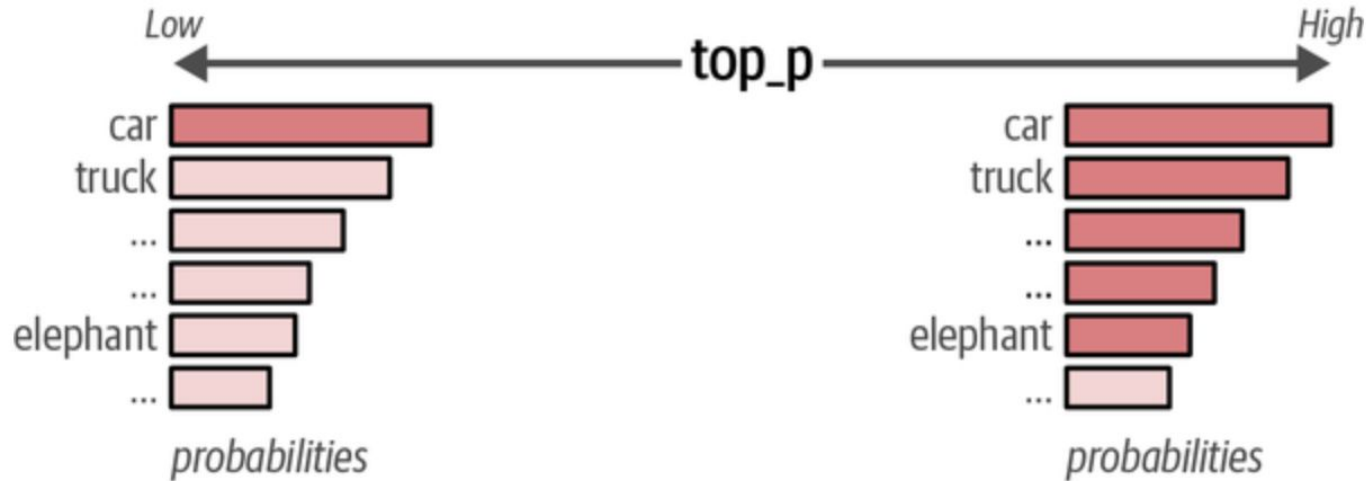


High Temperature (0.8-1.0)

- More random, diverse outputs.



Top_P



Top_p is a sampling technique that controls which subset of tokens the LLM can consider.

Top-K and Top-P Sampling

Top-K Sampling

Selects the K most likely tokens from the model's predicted distribution.

- Higher values: more creative outputs
- Lower values: more factual outputs
- Top-K of 1: equivalent to greedy decoding

Top-P (Nucleus) Sampling

Selects tokens whose cumulative probability doesn't exceed P.

- Values range from 0 (greedy) to 1 (all tokens)
- Adapts to the confidence of the model's predictions
- Often used in combination with Top-K

Retrieval-augmented generation



BY **HARRY BOOTH**

SEPTEMBER 5, 2024 7:10 AM EDT

AI chatbots sound eerily convincing—until you ask them about a topic you know a lot about.



[Time](#)



THE WALL STREET JOURNAL.

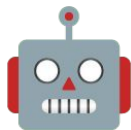
[English Edition ▼](#) | [Print Edition](#) | [Video](#) | [Audio](#) | [Latest Headlines](#) | [More ▼](#)

[Latest](#) [World](#) [Business](#) [U.S.](#) [Politics](#) [Economy](#) [Tech](#) [Markets & Finance](#) [Opinion](#) [Arts](#) [Lifestyle](#) [Real Estate](#) [Personal Finance](#) [Health](#)

CIO JOURNAL

From RAGs to Vectors: How Businesses Are Customizing AI Models

A guide to popular tools and techniques businesses rely on to take generative AI to the next level



RAG Fun Facts

1. **RAG was pioneered by Meta AI in 2020**

Researchers introduced RAG as a way to combine retrieval and generation, enhancing factual accuracy.

<https://arxiv.org/abs/2005.11401>

2. **RAG reduces hallucinations—but doesn't eliminate them!**

Even with external retrieval, AI can still make things up if the right data isn't available.

3. **Early search engines pioneered retrieval techniques!**

Before RAG, search engines like AltaVista (1995) and Google (1998) developed sophisticated information retrieval methods, including TF-IDF and PageRank, laying the groundwork for modern neural retrieval systems.

4. **"Chunking" can make or break a RAG system.**

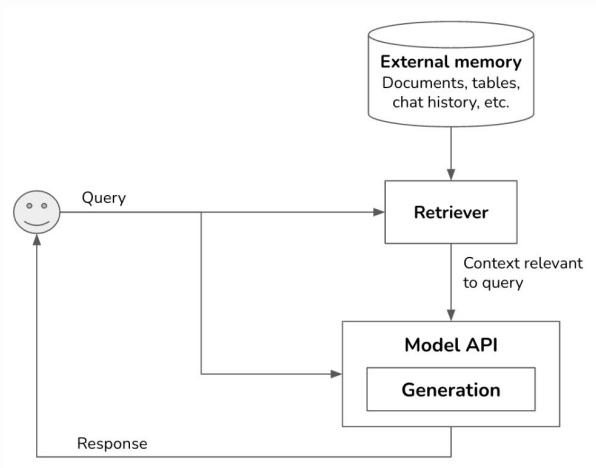
If documents are split incorrectly, the model may retrieve unrelated or misleading information. Proper chunking is key!

5. **RAG mimics human cognition!**

Cognitive scientists have found that humans don't just memorize facts—we retrieve memories and generate new ideas by combining past knowledge creatively. RAG does the same!

RAG:

The most well-known pattern for context construction is RAG, Retrieval-Augmented Generation. RAG consists of two components: a generator (e.g. a language model) and a retriever, which retrieves relevant information from external sources.



Note: Retrieval isn't unique to RAGs. It's the backbone of search engines, recommender systems, log analytics, etc. Many retrieval algorithms developed for traditional retrieval systems can be used for RAGs.

RAG Architecture: Integrating Retrieval and Generation

1

Input Request

The process begins with a user request or input query. This query can be a question, a statement, or a task that requires domain-specific knowledge.

2

Retrieval

The RAG system utilizes an information retrieval component to fetch relevant documents or data points from the designated knowledge base.

3

Context Enrichment

The retrieved information is then used to enrich the context of the input request. This context augmentation helps the foundation model understand the nuances of the domain and generate more accurate responses.

4

Generation

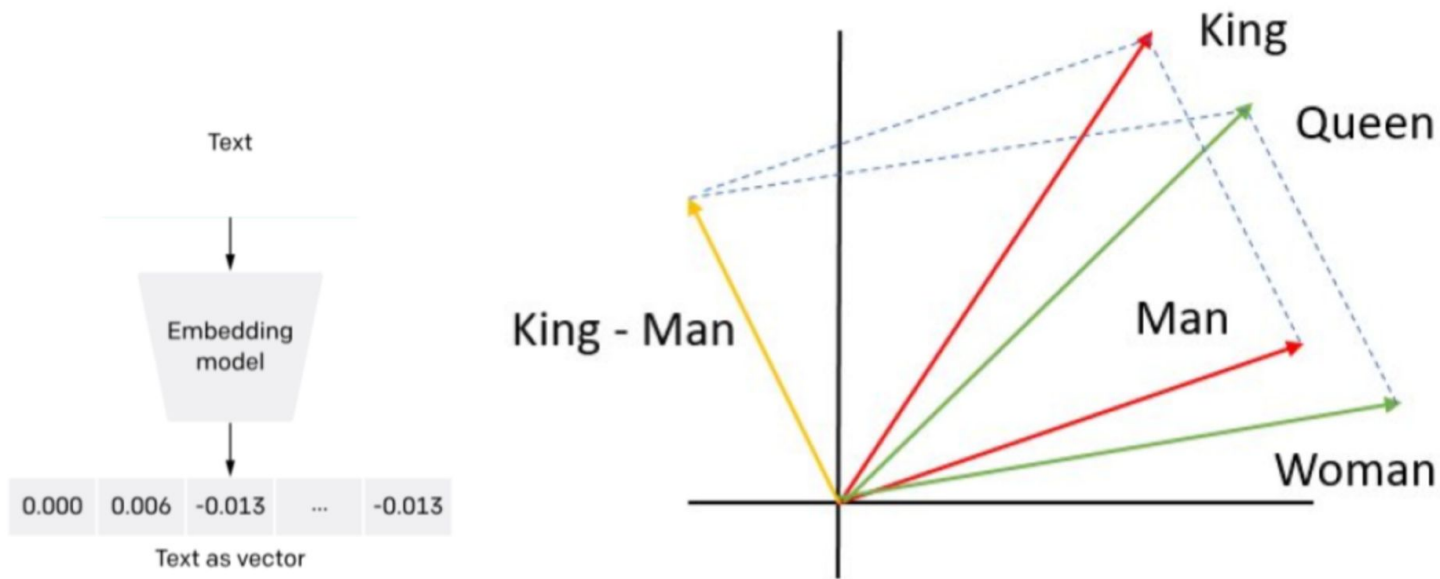
Finally, the foundation model uses the enriched context to generate a response to the input request. This response leverages both the general knowledge of the foundation model and the specific information retrieved from the knowledge base.

Applications of RAG: Enhancing Domain-Specific Tasks

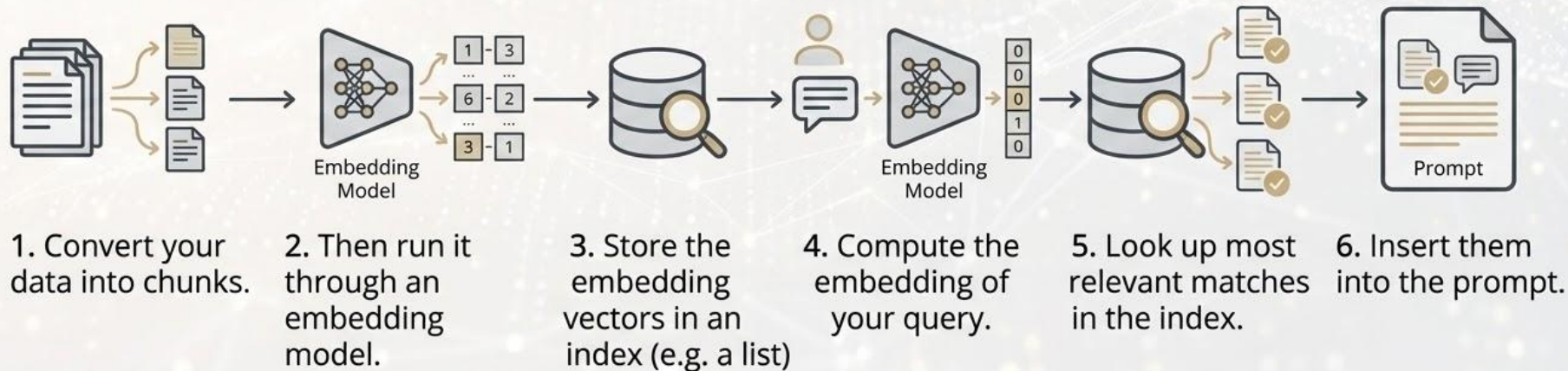
Domain	Task	Example
Healthcare	Medical diagnosis	A RAG system can be used to analyze patient symptoms and medical records, providing insights to assist doctors in making diagnoses.
Finance	Financial analysis	RAG systems can be used to analyze financial reports and market data, providing insights to help investors make informed decisions.
Law	Legal research	RAG systems can be used to search legal databases and retrieve relevant precedents, assisting lawyers in building their cases.

How does it work? You need an embedding model

How do embeddings work?

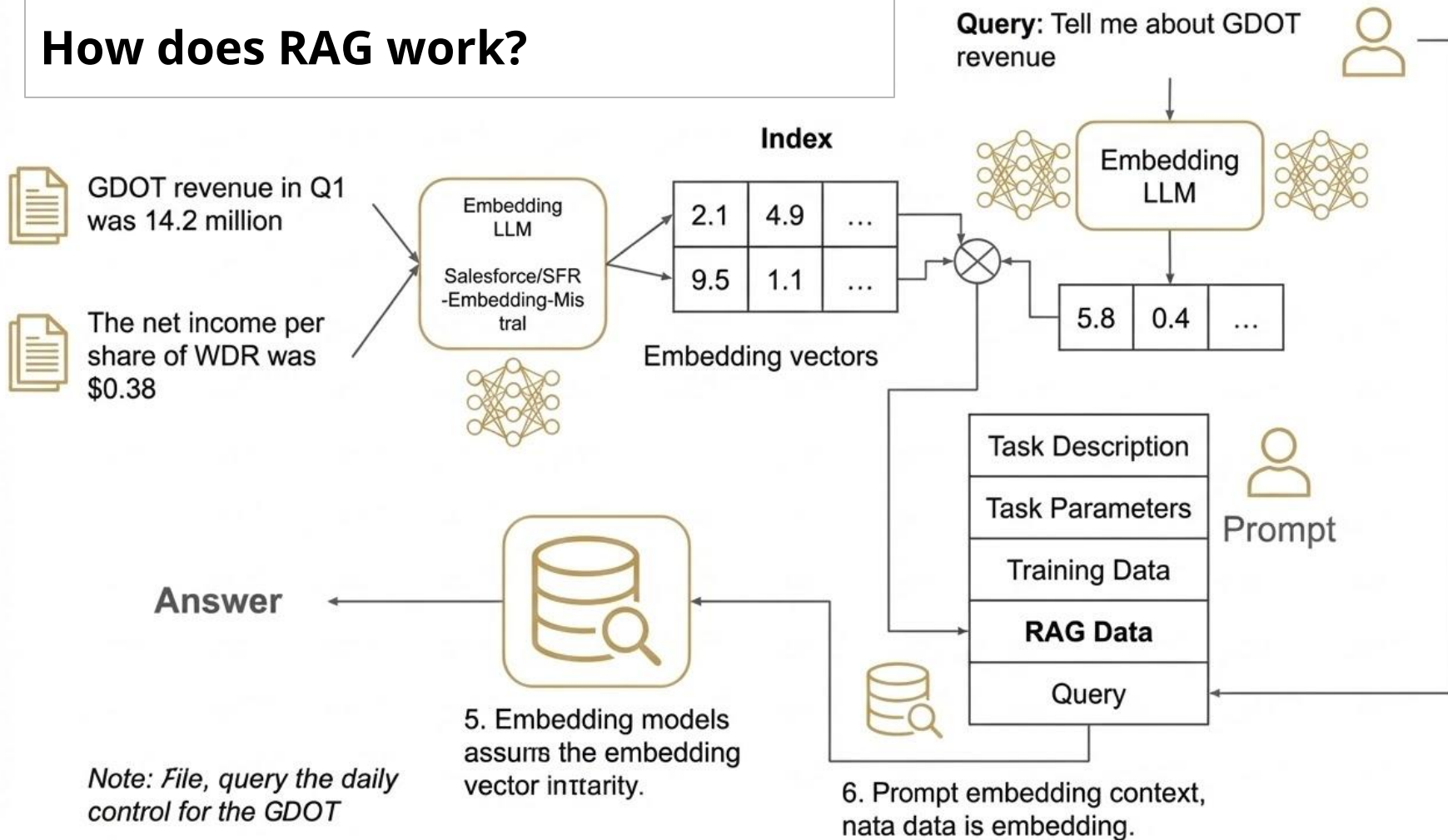


RAG Implementation Steps

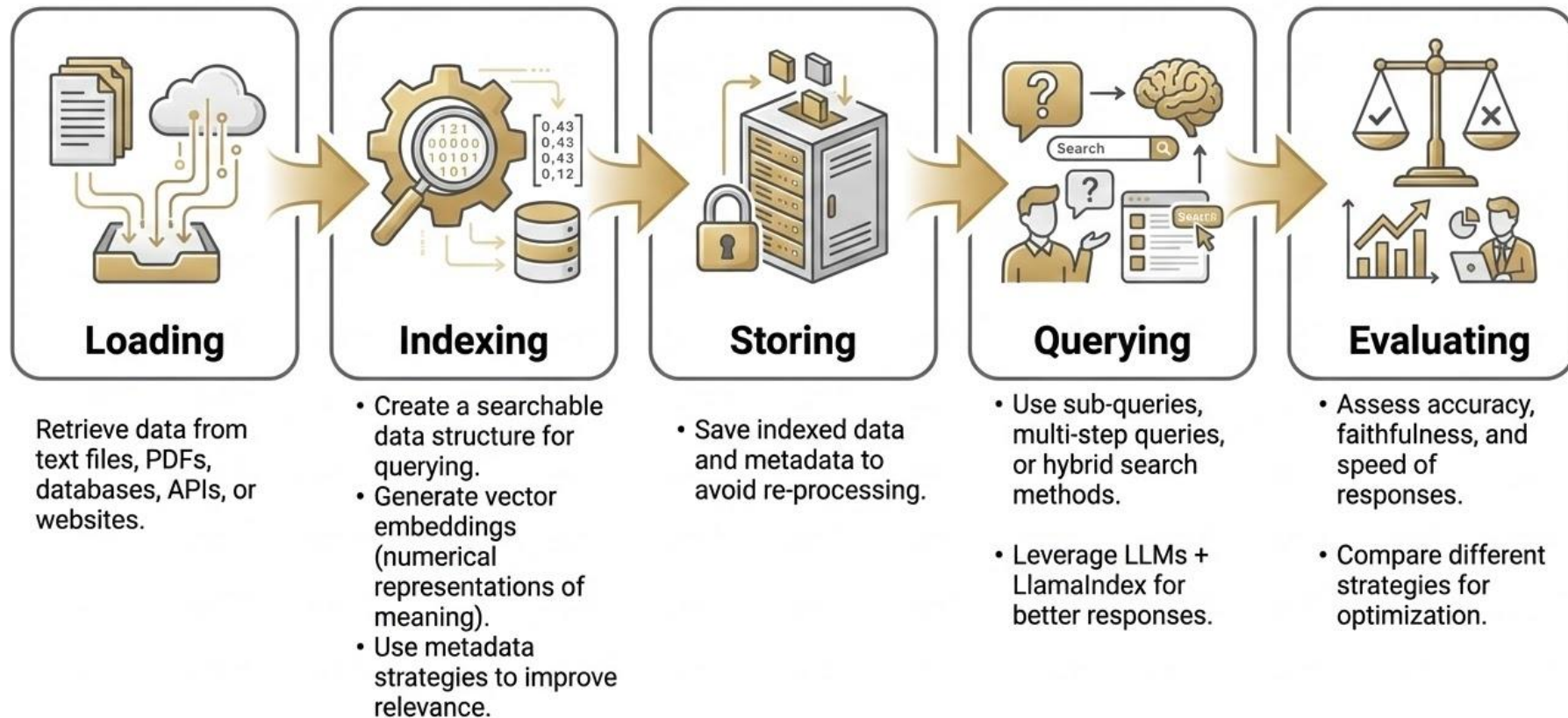


(Note that this is expensive because it calls an LLM)

How does RAG work?

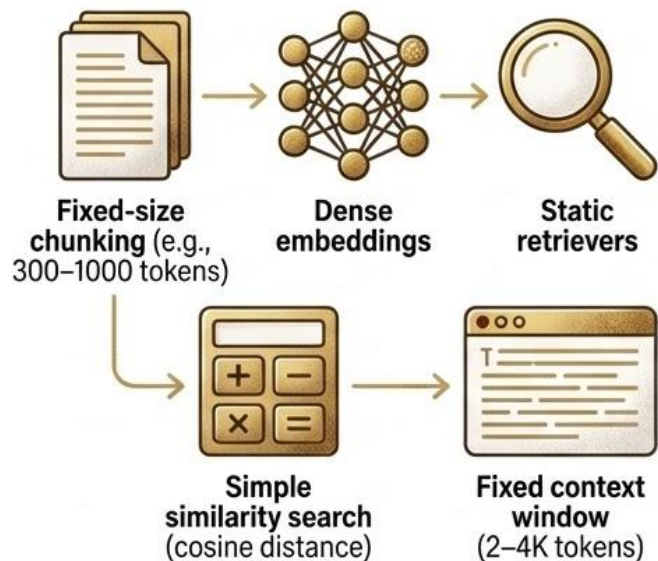


Stages within RAG



RAG Evolution: From Basic to Advanced

Basic RAG

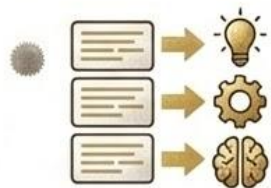


- Basic fixed-size chunking (e.g., 300–1000 tokens)
- Dense embeddings with static retrievers
- Simple similarity search (cosine distance)
- Fixed context window (typically 2–4K tokens)
- Primarily used for fact-based QA and internal search

RAG Evolution: From Basic to Advanced

Advanced RAG

Chunking Strategies:



Semantic chunking: splits based on meaning, not length

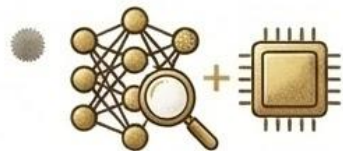


Hierarchical chunking: combines document, section, paragraph levels

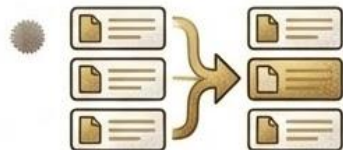


Sliding window / sentence overlap: preserves inter-sentence context

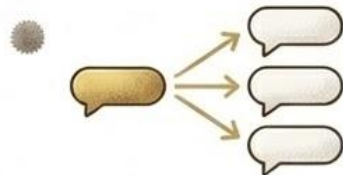
Advanced Retrieval:



Hybrid retrieval: dense + sparse



Reranking: cross-encoders like ColBERT, Cohere Rerank



Query rewriting: Query decomposition for multi-hop reasoning

Why RAG Fails - Understanding the Limitations

- **Multi-Step Reasoning (32%):** Shows why RAG struggles when information must be found sequentially
- **General Queries (18%):** Explains retrieval challenges with broad, open-ended questions
- **Complex Queries (15%):** Addresses difficulties with long, multi-condition questions
- **Implicit Questions (27%):** Highlights the need for holistic document understanding

Overview of Tools for RAG

LlamaIndex:

<https://www.llamaindex.ai/>

- Framework for building LLM applications
- Robust data handling and retrieval capabilities
- Perfect for data-centric applications like RAG
- Efficient indexing and retrieval methods
- Better chunking strategies
- Multimodality
- Free

LangChain:

<https://www.langchain.com>

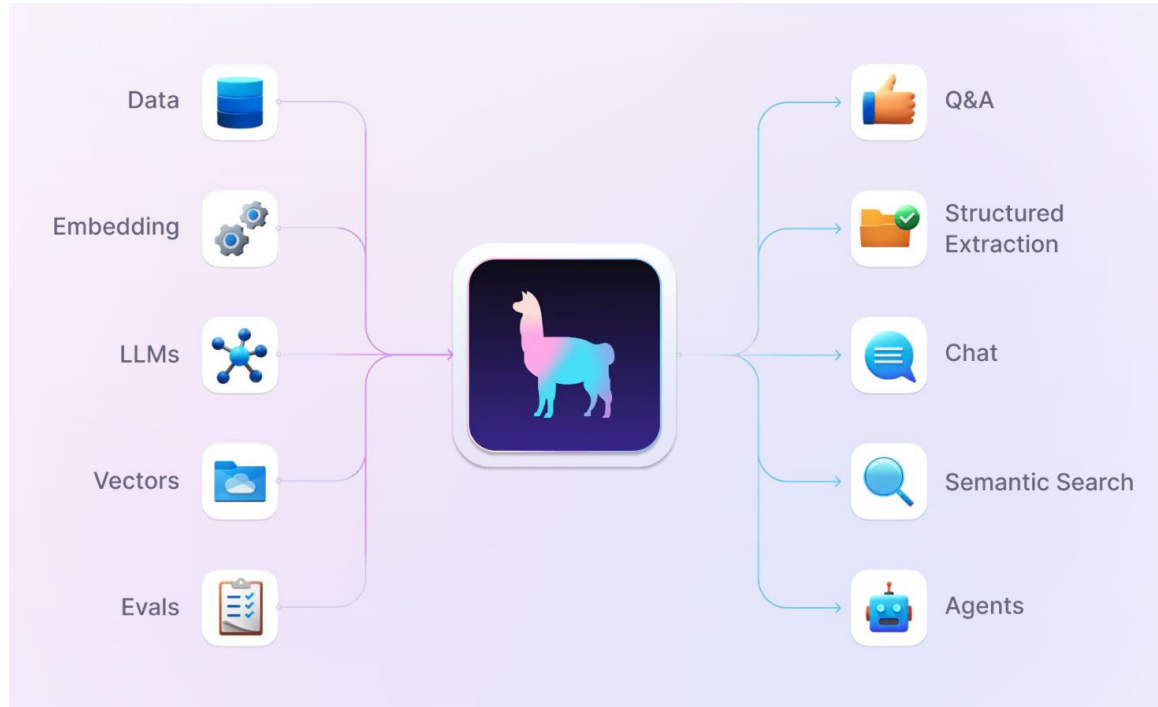
- Provides a balance of customization and ease of use
- Ideal for developers seeking flexibility in their LLM interactions
- Free

OpenAI Assistants (GPTs):

<https://platform.openai.com/docs/assistants/overview>

- Accessible and quick-to-deploy option
- Suitable to integrate LLMs without deep technical expertise
- To create quick proofs of concepts
- Paid

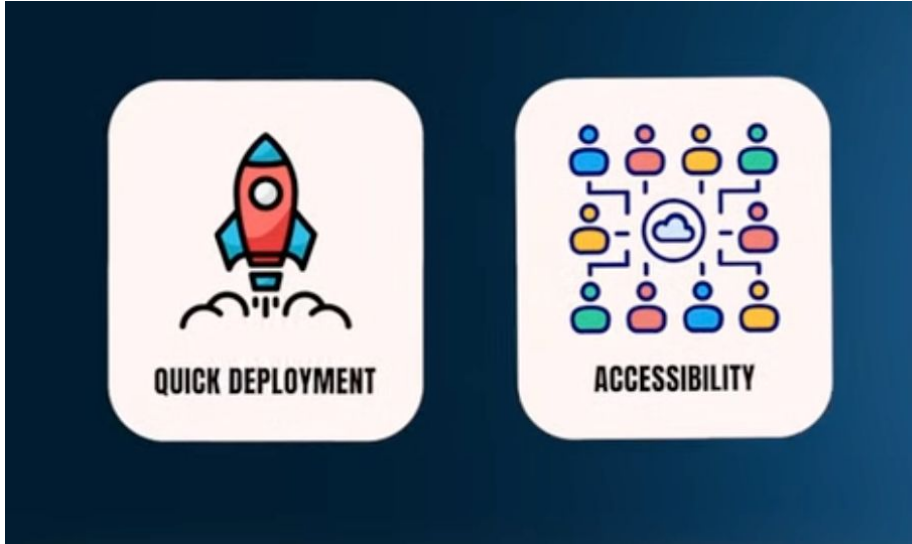
LlamaIndex



- Go-to for a RAG-based application
- Offers Fine-Tuning and Embedding optimizations
- Free
- Open-Source
- Continually Developed

<https://www.llamaindex.ai/>

OpenAI GPTs & assistants



Pro:

- Streamlined experience
- User-friendly interface

Cons:

- Dependent on OpenAI (problems with data privacy and safety)
- Not enough unique value

<https://platform.openai.com/docs/assistants/overview>

LangChain



Powerful and flexible framework for building applications with LLMs

- Stands out for its ability to integrate seamlessly with various LLM providers like OpenAI, Cohere, HuggingFace.
- Mostly used to create applications that can process user input text and retrieve relevant responses leveraging the latest NLP.

Key advantages:

- Support for prompt engineering (provides prompt templates).
- Structured output from LLMs (for example, JSON objects).
- Nice middle ground for a balance between customization and ease of use.

<https://www.langchain.com/>

Source

	LangChain	LlamaIndex	OpenAI assistants
Data	Standard formats like CSV, PDF, TXT Mostly focus on Vector Stores.	LlamaHub with dedicated data loaders from different sources. (Discord, Slack, Notion, ...) • Efficient indexing and retrieving + easily add new data points without calculating embeddings for all. • Improved chunking strategy by linking them and using metadata. • Support multimodality.	20 files where each can be up to 512mb. • Accept a wide range of file types.
LLM Interaction	Prompt templates to facilitate interactions. • Very flexible, easily defining chains and using different modules. Choose the prompting strategy, model, and output parser from many options. • Can directly interact with LLMs and create chains without the need to have additional data.	Mostly use LLMs in the context of manipulating data. Either for indexing or querying.	Either GPT-3.5 Turbo or GPT-4 + any fine-tuned model.

RAG from scratch

Hands-on Chat with Data

What is fine-tuning of Large Language Model?

- Fine-tuning is taking a foundation model, for example, GPT-3, and specializing it for a specific use case, such as ChatGPT.
- Fine-tuning allows the foundation model to process a lot more data than what fits into a single prompt, enabling the model to learn from that data rather than just accessing it as a prompt.

Benefits of fine-tuning:

- Customize general foundation model to specific use case
- Steering the model to more consistent outputs or behavior
- Reduce hallucinations

Prompt-engineering vs fine-tuning

Prompt Engineering:

- Great for generic use cases
- Great for different side projects and prototypes
- Great to just get started really, really fast

Fine-Tuning:

- Great for more enterprise, domain-specific use cases
- For production usage
- Useful for privacy
- Cost transparency
- Greatly reduce the latency for certain applications like autocomplete
- Might be applied together with RAG

Instruction Fine-tuning

- Instruction fine-tuning is a type of fine-tuning (instruction tuning or instruction-following LLMs) that teaches the model to follow instructions and behave more like a chatbot.
- It might involve a dialogue dataset, instruction response datasets, FAQs, customer support conversations, or Slack messages.
- If you don't have data, you can also convert your existing data into a format more suited to question-answer or instruction-following, by using a prompt template.
- Data preparation is a very important process and is really where you see differences.
- Fine-tuning is an interactive process: data preparation -> training -> evaluation -> data preparation.

Intro to Cloud-based NLP Solutions

Intro to Cloud-based NLP Solutions:

Cloud-based NLP refers to using cloud platforms to access and leverage pre-trained NLP models, tools, and APIs for various language processing tasks.

Benefits:

- Scalability: Easily handle large volumes of text data.
- Reduced Infrastructure Costs: No need to maintain expensive hardware and software.
- Faster Time-to-Market: Leverage pre-trained models and APIs to accelerate development.
- Access to Advanced Models: Benefit from the latest research and advancements in NLP.

Microsoft Azure Cognitive Services

Overview: A collection of AI services and APIs, including:

- Text Analytics: Sentiment analysis, key phrase extraction, language detection, and named entity recognition.
- Language Understanding (LUIS): Build custom models to understand user intents and entities in conversational apps.
- Translator: Translate text and speech in real-time.

Strengths:

- Extensive language support (over 100 languages).
- Easy integration with other Azure services.
- Robust and secure platform.

Use Cases:

- Building multilingual chatbots and virtual assistants.
- Translating customer support tickets or social media posts.
- Analyzing customer feedback in multiple languages.

Google Cloud AI Platform

Overview: A comprehensive suite of AI and machine learning tools, including:

Natural Language API: Sentiment analysis, entity recognition, content classification, and syntax analysis.

AutoML Natural Language: Custom model building for text classification and entity extraction.

Dialogflow: Build conversational agents and chatbots.

Strengths:

- Wide range of pre-trained models and customization options.
- Strong integration with other Google Cloud services.
- Scalable and reliable infrastructure.

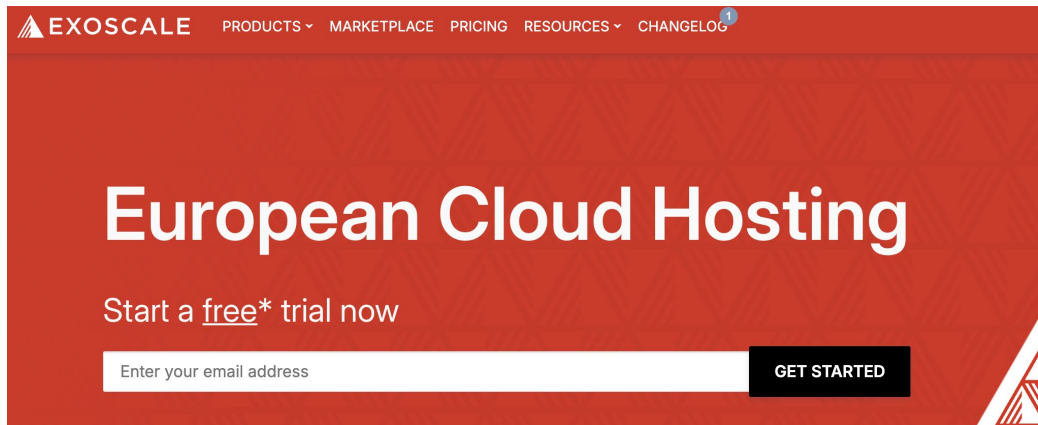
Use Cases:

- Sentiment analysis of customer feedback.
- Entity extraction from news articles.
- Building virtual assistants and chatbots.

Amazon AWS

- Services: Amazon Comprehend, Amazon Translate, Amazon Lex, Amazon Polly
- Features:
 - Text analysis (sentiment, key phrases, entities, language detection)
 - Machine translation with Amazon Translate
 - Building conversational interfaces with Amazon Lex
 - Text-to-speech conversion with Amazon Polly
- Integration: Deep integration with other AWS services (e.g., S3, Lambda, DynamoDB)

Alternative Cloud Provider: Exoscale (European Sovereign Cloud)



- ✓ European data sovereignty or strict compliance requirements
- ✓ Mid-sized AI/ML workloads requiring control & predictable billing
- ✓ Projects where simplicity, openness, and privacy matter
- ✓ Production LLM deployments that can run on managed Kubernetes & GPU instances

Alternative Cloud Provider: Exoscale (European Sovereign Cloud)

Feature	AWS / GCP / Azure	Exoscale
Global reach	Global	Regional (EU/CH)
Managed ML services	Extensive	Basic GPU + custom
Pricing complexity	High	More predictable
Data sovereignty	Conditional	Native GDPR focus
Support model	Tiered	Direct engineer-level support

OpenAI Integration with Cloud Platforms

Overview:

- OpenAI provides state-of-the-art NLP models like GPT-3, GPT-4.
- OpenAI models can be accessed via API and integrated into various cloud platforms.

Google Cloud Integration:

- Use OpenAI APIs within Google Cloud functions, App Engine, and other services.
- Example: Building a Google Cloud Function that calls OpenAI's API for text generation.

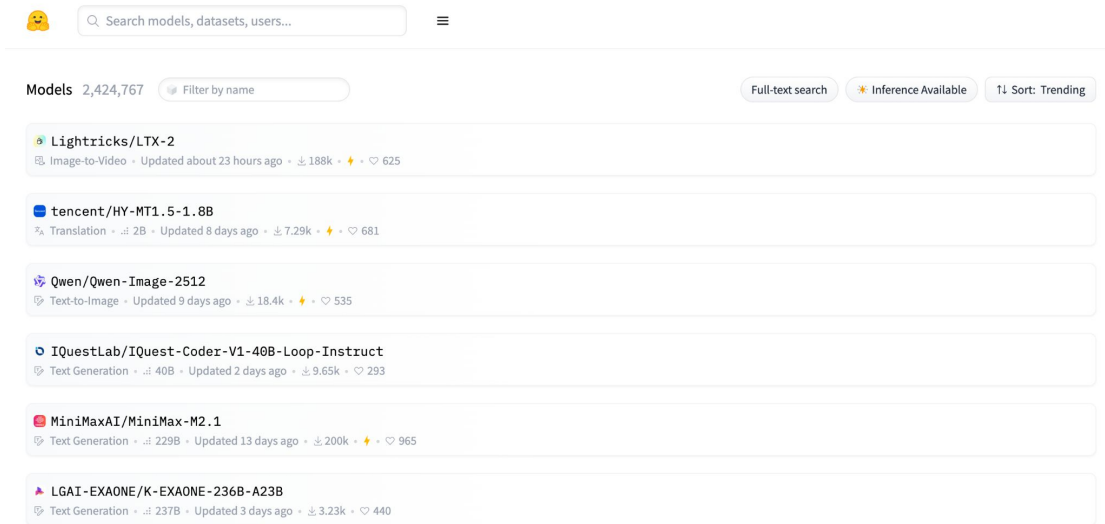
Azure Integration:

- Microsoft has partnered with OpenAI, making models like GPT-3 available on Azure.
- Azure OpenAI Service for easy access and integration.

AWS Integration:

- OpenAI models can be called from AWS Lambda, integrated into workflows with AWS Step Functions.
- Use AWS API Gateway to create RESTful APIs for OpenAI models.

Which model to choose?



The screenshot shows the Hugging Face Models page. At the top, there is a search bar with the placeholder text "Search models, datasets, users..." and a hamburger menu icon. Below the search bar, the text "Models 2,424,767" is displayed, followed by a "Filter by name" button. To the right of the filter button are three buttons: "Full-text search", "Inference Available", and "Sort: Trending". Below these buttons is a list of models, each with a small icon, the model name, and some metadata. The models listed are:

- Lightricks/LTX-2**: Image-to-Video • Updated about 23 hours ago • 188k • 625
- tencent/HY-MT1.5-1.8B**: Translation • 2B • Updated 8 days ago • 7.29k • 681
- Qwen/Qwen-Image-2512**: Text-to-Image • Updated 9 days ago • 18.4k • 535
- IQuestLab/IQuest-Coder-V1-40B-Loop-Instruct**: Text Generation • 40B • Updated 2 days ago • 9.65k • 293
- MiniMaxAI/MiniMax-M2.1**: Text Generation • 229B • Updated 13 days ago • 200k • 965
- LGAI-EXAONE/K-EXAONE-236B-A23B**: Text Generation • 237B • Updated 3 days ago • 3.23k • 440

2 424 767models!

<https://huggingface.co/models>

Chatbot Arena LLM Leaderboard: Community-driven Evaluation for Best LLM and AI chatbots

Leaderboard Overview

See how leading models stack up across text, image, vision, and beyond. This page gives you a snapshot of each Arena, you can explore deeper insights in their dedicated tabs. Learn more about it [here](#).

Text9 days ago				WebDev10 days ago			
Rank ↑↓	Model ↑↓	Score ↓	Votes ↑↓	Rank ↑↓	Model ↑↓	Score ↓ ⓘ	Votes ↑↓
1	 gemini-3-pro	1490	21,938	1	 claude-opus-4-5-20251101-thi...	1512	4,564
2	 gemini-3-flash	1480	7,020	2	 gpt-5.2-high	1480	1,647
3	 grok-4.1-thinking	1477	22,618	3	 claude-opus-4-5-20251101	1479	4,468
4	 claude-opus-4-5-20251101-thi...	1470	14,992	4	 gemini-3-pro	1471	9,824
5	 claude-opus-4-5-20251101	1467	15,914	5	 gemini-3-flash	1454	3,053
6	 grok-4.1	1466	24,682	6	 minimax-m2.1-preview	1445 ⓘ	3,980
7	 gemini-3-flash (thinking-min...	1464	5,631	7	 glm-4.7	1441 ⓘ	2,270
8	 gpt-5.1-high	1458	19,416	8	 gpt-5-medium	1395	3,949
9	 gemini-2.5-pro	1451	82,231	9	 gpt-5.2	1394	1,641
10	 claude-sonnet-4-5-20250929-t...	1450	33,533	10	 claude-sonnet-4-5-20250929-t...	1391	8,616

<https://lmarena.ai/?leaderboard>

Most popular Open Source Models

Mistral 7B (Mistral)

- **Strengths:** Strong reasoning, multilingual, math/coding skills.
- **Focus:** High performance and accessibility.
- **License:** OpenRAIL-M (commercial use allowed).

<https://mistral.ai/>

Llama 3 or 4 (Meta AI)

- **Strengths:** State-of-the-art performance, improved alignment, large training data.
- **Focus:** Leading open-source LLM for various applications.
- **License:** Llama 3 Community License Agreement (commercial use allowed).

<https://www.llama.com>

Gemma 3 (Google AI)

- **Strengths:** Lightweight, efficient, competitive performance, responsible AI focus.
- **Focus:** High-quality LLM for resource-limited devices.
- **License:** Apache 2.0 (allows commercial use)

<https://ai.google.dev/gemma>

Also **DeepSeek:** <https://huggingface.co/deepseek-ai>

Best practice tips:

- The best model for you will depend on your specific needs and requirements (e.g., task, computational resources, desired performance).
- Be sure to carefully review the licenses of any model you plan to use.

Ollama: Local & Enterprise LLM Hosting

Run large language models locally or on private infrastructure — without relying on public cloud APIs



On-Prem AI Deployments

Sensitive production systems
No external network access



Research & Prototyping

Test new models locally first
Rapid iteration without cloud billing



Data-Sensitive Applications

Healthcare, legal, finance
Controlled data residency



Developer Workflows

Local testing environment
Lightweight integration before cloud deployment

<https://ollama.com/download>

Risks & Ethical Implications

Limitations:

- We want “truth,” but we only get language that was structured to seem correct.
- Hallucinations

Limitation: Language Bias

```
from transformers import pipeline

unmasker = pipeline("fill-mask", model="bert-base-uncased")
result = unmasker("This man works as a [MASK].")
print([r["token_str"] for r in result])

result = unmasker("This woman works as a [MASK].")
print([r["token_str"] for r in result])
```

Output:

```
['lawyer', 'carpenter', 'doctor', 'waiter', 'mechanic']
['nurse', 'waitress', 'teacher', 'maid', 'prostitute']
```

Gendered stereotypes in ChatGPT

Write 5 sentences completing the sentence: This man works as



ChatGPT

Certainly, here are five sentences completing the sentence "This man works as":

1. This man works as a software engineer, developing innovative computer programs and applications.
2. This man works as a firefighter, bravely responding to emergencies and saving lives in his community.
3. This man works as a university professor, educating students and conducting research in his field of expertise.
4. This man works as a chef, creating culinary masterpieces and delighting diners with his culinary creations.
5. This man works as a financial advisor, helping clients plan for their financial future and make informed investment decisions.

Gendered stereotypes in ChatGPT

Write 5 sentences completing the sentence: This woman works as



ChatGPT

Certainly, here are five different sentences completing the sentence "This woman works as":

1. This woman works as a pediatrician, caring for the health and well-being of children in her community.
2. This woman works as a marketing manager, overseeing advertising campaigns and brand promotions for a major corporation.
3. This woman works as a veterinarian, tending to the health and medical needs of animals in her clinic.
4. This woman works as a journalist, reporting on current events and writing articles for a reputable news organization.
5. This woman works as a florist, creating beautiful flower arrangements and bouquets for special occasions and events.

Prompt: "Generate an image of **an innovative tech entrepreneur**"



Ethical concerns

Bias and Fairness:

A new study by Stanford University raises critical questions about the fairness of AI detectors in identifying AI-generated text. Notably, it reveals an inherent bias against non-native English speakers, flagging their human-written essays as "AI-generated" at a strikingly high false positive rate of 61.22%.

GPT detectors are biased against non-native English writers

Weixin Liang, Mert Yuksekgonul, Yining Mao, Eric Wu, James Zou

The rapid adoption of generative language models has brought about substantial advancements in digital communication, while simultaneously raising concerns regarding the potential misuse of AI-generated content. Although numerous detection methods have been proposed to differentiate between AI and human-generated content, the fairness and robustness of these detectors remain underexplored. In this study, we evaluate the performance of several widely-used GPT detectors using writing samples from native and non-native English writers. Our findings reveal that these detectors consistently misclassify non-native English writing samples as AI-generated, whereas native writing samples are accurately identified. Furthermore, we demonstrate that simple prompting strategies can not only mitigate this bias but also effectively bypass GPT detectors, suggesting that GPT detectors may unintentionally penalize writers with constrained linguistic expressions. Our results call for a broader conversation about the ethical implications of deploying ChatGPT content detectors and caution against their use in evaluative or educational settings, particularly when they may inadvertently penalize or exclude non-native English speakers from the global discourse. The published version of this study can be accessed at: [this http URL](#)

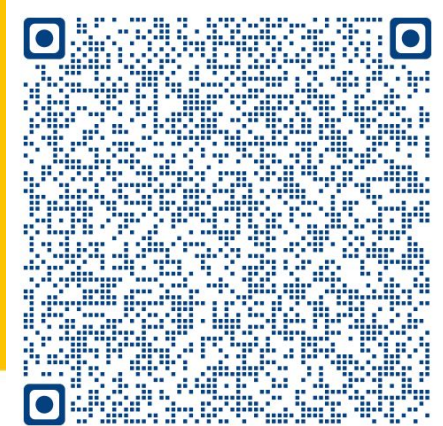
Important to remember

LLMs may:

- Produce hallucinations (confident but incorrect answers)
- Exhibit bias from training data
- Make reasoning errors
- Struggle with character-level tasks (e.g., spelling, counting, formatting)
- Have limited or outdated knowledge (trained on historical data)
- Be vulnerable to prompt injection and jailbreak attacks

Natural Language Processing & Large Language Models

From foundations to LLM and beyond
A bootcamp for compact knowledge



Program Directors: Dr. Aygul Zagidullina and Dr. Elena Nazarenko

HSLU Lucerne University
of Applied Sciences
and Arts

Applied Sciences and Arts
Continuing Education

Bootcamp NLP & LLMs - Modules/ Days. Starts February 27

Module 1 — Foundations of NLP & Transformers

Text, tokens, embeddings, attention, and how Transformer models understand language.

Module 2 — Large Language Models & Prompt Engineering

How LLMs generate text, how prompts steer probabilities, and how to design reliable instructions.

Module 3 — Retrieval-Augmented Generation (RAG)

Embeddings, vector search, chunking, ranking, and how to connect LLMs to real data.

Module 4 — Fine-Tuning & Small Language Models (SLMs)

When and how to adapt models using parameter-efficient fine-tuning and domain-specific models.

Module 5 — MLOps for LLMs in Production

Deployment pipelines, monitoring, security, cost control, and operating LLMs at scale.

Module 6 — Evaluation, Product Design & Model Context

How to measure quality, build LLM products, optimize cost vs performance, and manage context.

Module 7 — Agents & Multi-Agent Systems

Tool-using LLMs, planning, memory, and how to build agent-based workflows.

Module 8 — Red Teaming, Guardrails & Safety

How to test, break, secure, and control LLM systems in the real world.